

Software Requirements Specification (SRS)

Project: SurveyMarket

Version: 2.0 Date: May 2026 Status: Approved

Table of Contents

- [1. Introduction](#)
 - [2. Overall Description](#)
 - [3. External Interface Requirements](#)
 - [4. System Features](#)
 - [5. Nonfunctional Requirements](#)
 - [6. Regulatory & Compliance Requirements](#)
-

1. Introduction

1.1 Purpose

The purpose of this document is to provide a comprehensive, highly detailed Software Requirements Specification (SRS) for **SurveyMarket**, a full-stack, dual-sided SaaS survey marketplace platform. This document describes the system architecture, functional capabilities, non-functional constraints, and interface specifications to guide the development, testing, and deployment phases.

1.2 Document Conventions

- Prioritization:** Requirements are categorized as High (Must Have), Medium (Should Have), and Low (Nice to Have).
- Typography:** Bold text indicates critical features or database entities. Monospace font is used for code snippets, API endpoints, and configuration parameters.

1.3 Intended Audience and Reading Suggestions

This SRS is intended for:

- Development Team:** Frontend (Next.js) and Backend (FastAPI) developers.
- Project Managers & Stakeholders:** To track feature completeness.
- QA Team:** To formulate test plans and test cases.
- Compliance Officers:** To verify adherence to the DPDP (Digital Personal Data Protection) Act of India.

1.4 Product Scope

SurveyMarket is designed to seamlessly connect researchers (companies) with target demographics (users/respondents).

- Companies can fund their wallets, design comprehensive surveys, and define target criteria.
- Users can securely register, browse eligible surveys, provide their data in exchange for monetary compensation, and withdraw their earnings.
- The platform automatically calculates survey budgets including a 10% platform commission, reserves the funds, and releases them to respondents instantaneously upon successful survey completion.

- The entire system is enveloped in an Apple-inspired, premium user interface characterized by minimalistic typography, glassmorphism, and fluid micro-animations.

2. Overall Description

2.1 Product Perspective

SurveyMarket is a standalone web application utilizing a microservices-inspired monolithic architecture. It replaces manual survey distribution methods with an automated marketplace.

Component	Technology	Description
Frontend UI/UX	Next.js 14, React, Tailwind CSS	Client-side rendering & SSR for responsive dashboards
Backend API	FastAPI, Python 3.11	High-performance, async RESTful API serving client requests
Database	PostgreSQL 15, SQLAlchemy 2.0	Relational database mapping for complex user/survey schema
Cache & Queues	Redis 7, Celery 5	Background task processing, caching, and rate limiting
Payments Integration	Razorpay API	Processing company deposits and user withdrawals securely
Containerization	Docker, Docker Compose	Consistent deployment across development and production

2.2 User Classes and Characteristics

User Class	Characteristics	Primary Use Cases	Privilege Level
Respondent (User)	General internet users. Varies from students to professionals. Motivated by monetary rewards.	Browsing surveys, submitting answers, withdrawing earnings, updating profile for better targeting.	Low
Researcher (Company)	Market researchers, product managers, academia. Focus on data accuracy and rapid insights.	Depositing funds, building surveys, analyzing survey results, downloading CSV reports.	Medium
Administrator (Admin)	Platform owners/operators. Highly technical or operational staff.	Approving/Rejecting surveys, moderating user activity, processing withdrawals, viewing system analytics.	High (Root)

2.3 Operating Environment

- **Client Requirements:** Any modern web browser (Google Chrome v90+, Safari v14+, Firefox v88+). Must have JavaScript enabled. Responsive down to 320px width (mobile phones).
- **Server Requirements:** Linux-based OS (Ubuntu 22.04 LTS), Minimum 4GB RAM, 2 vCPUs for backend containers.

2.4 Design and Implementation Constraints

- **Regulatory Constraint:** Complete alignment with the Indian DPDP Act. This mandates explicit user consent, purpose limitation, data minimization, and the implementation of a "Right to be Forgotten".
- **Financial Constraint:** The wallet logic must use transactional atomicity (ACID properties) to completely prevent race conditions that could lead to double-spending or negative balances.
- **Design Constraint:** Usage of predefined design tokens for an "Apple-inspired" aesthetic. Strict adherence to fluid animations (Framer Motion) and high-fidelity visual assets.

3. External Interface Requirements

3.1 User Interfaces

The system shall present three primary dashboards:

1. **Landing Page:** High-conversion funnel, premium animations, explaining value propositions for both companies and users.
2. **User Dashboard (/dashboard/*):**
 - **Available Surveys View:** Grid/List of survey cards showing reward amount, estimated time, and category.
 - **Wallet View:** Current balance, transaction history, withdrawal request form.
3. **Company Dashboard (/dashboard/company/*):**
 - **Survey Builder View:** Drag-and-drop or form-based interface to add multiple-choice, text, or rating questions.
 - **Analytics View:** Charts (Recharts) detailing survey completion rates and answer distribution.
4. **Admin Panel (/admin/*):** Data tables with actions to approve/reject surveys and process withdrawal queues.

3.2 Software Interfaces

- **Razorpay API:** Used for capturing payments. Webhook endpoint (/company/payment/verify) must securely verify the HMAC signature of all Razorpay events before updating database balances.
- **Google OAuth 2.0:** Interfaces with https://accounts.google.com for SSO login. Handled via backend httpx client exchanging authorization codes for access tokens.

4. System Features

4.1 Feature 1: Authentication and Authorization Framework

- **Description:** Multi-role authentication system supporting standard credentials and Google SSO.
- **Priority:** High

Functional Details:

Action	Input Data	Processing Steps	Output / Result
--------	------------	------------------	-----------------

Registration	Email, Password, Role (User/Company)	1. Validate email format & password strength. 2. Check for duplicate email. 3. Hash password using bcrypt. 4. Save to users table. 5. Generate JWT.	HTTP 201 Created. Returns JWT Bearer token and user schema.
Login	Email, Password	1. Retrieve user. 2. Verify bcrypt hash. 3. Generate short-lived Access Token & Refresh Token.	HTTP 200 OK. Returns tokens.
Google Auth	Google OAuth Code	1. Validate state parameter. 2. Exchange code for Google token. 3. Fetch user profile. 4. Auto-register if new, else login.	HTTP 200 OK. Redirects to frontend with platform JWT.

4.2 Feature 2: Survey Creation and Budgeting Engine

- **Description:** Tool for companies to construct surveys and allocate budgets.
- **Priority:** High

Functional Details:

Field / Action	Description	Validation / Constraints
Title & Description	Core metadata for the survey.	Title: Max 100 chars. Description: Max 500 chars.
Questions Array	JSON structure of questions.	Minimum 1 question. Supported types: text, choice, rating.
Reward per User	Monetary amount given to one respondent.	Minimum ₹10. Must be a positive decimal.
Target Responses	Number of users required to complete the survey.	Minimum 10 responses.
Budget Calculation	Auto-calculated field.	$(\text{Reward} \times \text{Target Responses}) \times 1.10$ (includes 10% platform fee).
Publishing Check	Validating company wallet balance before setting status to 'PENDING'.	$\text{Company_Wallet_Balance} \geq \text{Calculated_Budget}$. If True, deduct amount and mark survey pending admin review.

4.3 Feature 3: Survey Execution & Anti-Fraud

- **Description:** Mechanism for users to take surveys, enforcing integrity.

- **Priority:** High

Functional Details:

- **Single Submission Constraint:** Database-level unique constraint on (user_id, survey_id) in the submissions table.
- **IP & Speed Checking (Fraud Detection):**
 - **Rule 1:** If time spent on survey is < (Question_Count * 3 seconds) , flag submission as SUSPICIOUS .
 - **Rule 2:** If > 3 submissions from the same IP address within 1 hour, flag IP and rate-limit.
- **Wallet Crediting:** If submission is VALID , execute an atomic DB transaction to increment User_Wallet_Balance by Reward_Amount and decrement Survey_Remaining_Budget .

4.4 Feature 4: Financial Transactions & Wallet

- **Description:** Ledger system handling deposits and withdrawals.
- **Priority:** High

Functional Details:

Transaction Type	Trigger	System Action	Status Lifecycle
Deposit	Company clicks "Add Funds"	1. Call Razorpay API to create Order. 2. Client completes payment. 3. Webhook verifies signature. 4. Credit company wallet.	CREATED -> VERIFIED / FAILED
Withdrawal Request	User clicks "Withdraw"	1. Check if Balance >= ₹500 (Minimum threshold). 2. Deduct from User Wallet. 3. Create withdrawal ticket for Admin.	PENDING -> APPROVED (Admin executes off-platform payout) -> COMPLETED
Refund	Admin rejects survey OR survey expires	1. Calculate Unused_Budget = Remaining_Responses * Reward. 2. Add Unused_Budget + (Unused_Budget * 10%) back to Company Wallet.	REFUNDED

4.5 Feature 5: Background Jobs (Celery)

- **Description:** Asynchronous tasks to keep the system responsive.
- **Priority:** Medium

Tasks Executed:

1. **check_expired_surveys (Hourly):** Scans DB for surveys past their end date. Changes status to EXPIRED and triggers refund logic for unused budget.
2. **generate_daily_analytics (Nightly, 00:00 UTC):** Aggregates daily completions, active users, and platform revenue into an analytics table for dashboard rendering.

5. Nonfunctional Requirements

5.1 Performance Requirements

- **Latency:** Core API read operations (fetching surveys) must complete in under 200ms at the 95th percentile.
- **Concurrency:** The system must handle 500 concurrent users taking surveys without degraded performance, leveraging FastAPI's ASGI asynchronous event loop.
- **Database Indexing:** B-Tree indexes must be applied on `user_id`, `survey_id`, and `status` columns to ensure rapid query execution.

5.2 Security Requirements

- **Authentication:** All protected API endpoints must require a valid JWT passed via the `Authorization: Bearer <token>` header.
- **SQL Injection Prevention:** Enforced universally via SQLAlchemy ORM parameterized queries. Raw SQL is strictly prohibited.
- **XSS & CSRF:** Handled by Next.js built-in sanitization and proper CORS header configurations on the backend.
- **Rate Limiting:** Global rate limiting of 100 requests per minute per IP address applied using SlowAPI and Redis.

5.3 Reliability and Availability Requirements

- System architecture relies on Docker-Compose for self-healing mechanisms (`restart: unless-stopped`).
- PostgreSQL must have periodic backups scheduled.
- If Redis crashes, Celery tasks will be queued in memory or disk until the broker is restored.

5.4 Software Quality Attributes

- **Usability:** The UI must adhere to the W3C Web Content Accessibility Guidelines (WCAG) 2.1 Level AA. Forms must have clear error states and validation messages (using Zod & React Hook Form).
- **Maintainability:** Backend code must pass `flake8` and `mypy` static analysis. Frontend code must pass ESLint strict rules.

6. Regulatory & Compliance Requirements

6.1 DPDP Act (India) Compliance Matrix

The platform incorporates strict privacy measures tailored for the Digital Personal Data Protection Act:

DPDP Principle	Implementation in SurveyMarket
Explicit Consent	Users must check a mandatory, un-pre-checked consent box before registering and before starting any survey.
Purpose Limitation	Data collected during surveys is exclusively available to the company that created the survey and cannot be aggregated or sold by the platform to third parties.
Data Minimization	The platform only requires Email and Name for registration. Demographic data is optional and used solely for survey matching.

Right to Erasure	/user/account/delete endpoint provided. Executes a cascade delete of user profile, anonymizes survey submissions, and invalidates tokens.
Breach Notification	Admin dashboard includes a secure broadcast system to notify users within 72 hours in the event of a suspected data breach.

End of Document